

A Search-and-Learning Solver for the STL Handkerchief Game

Repository white-paper draft

May 28, 2026

Abstract

This draft documents the current solver architecture in the repository and states the mathematical contract that must hold for the solver to be called defensible. The implemented system models the handkerchief game as a finite, stochastic, zero-sum, perfect-information game with simultaneous move matrix games at each half-round. The code currently contains an engine-derived exact finite-horizon minimax solver, a selective exact-second search layer, tablebase pins, matrix-game Monte Carlo tree search (MCTS), a neural value-and-policy network, supervised target generation, an AlphaZero-style bootstrap loop, calibration gates, tournament evaluation, and audit-pack checks. The latest documented run, Phase G iteration 3, reaches held-out overall MSE 0.05934, a 45 percent improvement over the pre-Phase-G baseline. The remaining rigor gap is not the existence of a formal game value, but the approximation chain: finite horizon, selective candidate sets, sampled MCTS values, and learned value-network error.

1 Scope and Repository Snapshot

The repository has shifted from a playable baseline toward a Stockfish/AlphaZero-style solver. The central design choice is to keep the canonical game in a fixed public-state legality regime rather than introduce Bayesian type inference. In the current code, the strategic asymmetry is encoded directly in legality: Baku may drop on second 61 during the leap window, but Hal never may drop or check on second 61. This makes the solver a perfect-information stochastic-game solver with simultaneous actions, not a poker-style imperfect-information solver.

The last few weeks of commits can be summarized as follows.

- April 28 built the rigorous core: exact public states, LP minimax, engine-derived chance expansion, a firewall separating rigorous CFR code from shaping and bucketed abstractions, tablebase targets, selective exact-second search, and MCTS slices through selection, expansion, backup, search, transposition, and principal-line diagnostics.
- April 29–30 added the first value-target generation, the training harness, calibration metrics, tournament evaluation, subgame resolve, and the first bootstrap-loop API.
- May 18 hardened the bootstrap loop into a defensibility gate: stronger evaluator interfaces, audit packs, subgame-resolve hooks, source-weighted training, held-out calibration, and tests for legality, self-play, tournaments, and target generation.
- May 19–21 fixed corpus death-axis handling and CPR scaling, added corpus diagnostics, and expanded the pinned tablebase registry.
- May 24 added per-source calibration thresholds, a hidden-width architecture knob, a bounded LRU cache for exact LP recursion, and the Phase G wider LP-corpus script.

- May 26–27 recorded the Phase G + I-1 milestone and two subsequent bootstrap iterations. Iteration 3 converged at held-out MSE 0.05934.

The most important implementation files are:

Path	Role
<code>src/Game.py</code>	Authoritative half-round state machine
<code>src/Player.py</code> , <code>src/Referee.py</code>	Player and survival dynamics
<code>environment/legal_actions.py</code>	Actor-aware action legality
<code>environment/cfr/exact.py</code>	Exact public state, LP minimax, finite-horizon recursion
<code>environment/cfr/selective.py</code>	Candidate generation and selective minimax search
<code>environment/cfr/mcts.py</code>	Matrix-game-at-node MCTS
<code>environment/cfr/evaluator.py</code>	Terminal, tablebase, and value-net evaluators
<code>environment/cfr/tablebase.py</code>	Pinned tactical registry
<code>training/value_targets.py</code>	Exact and MCTS target generation
<code>training/train_value_net.py</code>	Supervised value/policy training
<code>training/bootstrap_loop.py</code>	Generation orchestration and hard gates
<code>training/calibration.py</code>	Held-out MSE, Brier score, reliability bins
<code>training/audit_pack.py</code>	Tactical audit records and drift gates
<code>hal/value_net.py</code>	Current neural architecture and 23-feature extractor

2 Canonical Game Model

2.1 Public State

Let the two players be $i \in \{\text{Hal}, \text{Baku}\}$. A public state $s \in \mathcal{S}$ is represented in the engine by the tuple

$$s = (x_{\text{Hal}}, x_{\text{Baku}}, \tau_{\text{Hal}}, \tau_{\text{Baku}}, d_{\text{Hal}}, d_{\text{Baku}}, a_{\text{Hal}}, a_{\text{Baku}}, k, t, h, r, f, g, w),$$

where x_i is the current cylinder time, τ_i is total prior time dead, d_i is the number of prior deaths, a_i records life status, k is referee CPR count, t is game-clock seconds since 8:00:00 AM, $h \in \{1, 2\}$ is the half-round, r is the round index, f is the first dropper, g records terminal status, and w records the winner when terminal. In code this is `environment/cfr/exact.py`'s `ExactPublicState`.

The canonical opening state uses $t = 720$, i.e. 8:12:00 AM. The leap window is $[3540, 3600]$ in engine seconds, corresponding to the 8:59 minute and the inserted second 8:59:60.

2.2 Half-Round Actions

Each half-round has a dropper $D(s)$ and checker $C(s)$. The action is a simultaneous pair

$$a = (d, c), \quad d \in A_D(s), \quad c \in A_C(s),$$

where d is the drop second and c is the check second. Normal turns have duration 60. Leap-window turns have duration 61.

The current actor-aware legality rule is:

$$\begin{aligned} A_D(s) &= \{1, \dots, 60\} && \text{if } D(s) = \text{Hal}, \\ A_D(s) &= \{1, \dots, T(s)\} && \text{if } D(s) = \text{Baku}, \\ A_C(s) &= \{1, \dots, 60\} && \text{for both players,} \end{aligned}$$

where $T(s) = 61$ only in the leap window and $T(s) = 60$ otherwise. This is the decisive model reduction: Hal cannot act on second 61, and Baku's only leap-second privilege is as dropper.

2.3 Deterministic Resolution Before Death Chance

For action $a = (d, c)$, the checker succeeds iff $c \geq d$. If the checker succeeds, squandered time is

$$\Delta x_C = \max(1, c - d).$$

If the check fails, the checker receives the fixed failed-check penalty

$$\Delta x_C = 60.$$

If the checker cylinder reaches or exceeds 300 seconds, or if the checker failed, the checker enters a death episode. The death duration is

$$z = \min(x_C + \Delta x_C, 300).$$

If no death occurs, the transition is deterministic. If a death occurs, the transition branches on survival.

2.4 Survival Probability

The referee computes survival probability as

$$p_{\text{survive}}(i, z, s) = \left[1 - \left(\frac{z}{300} \right)^3 \right]_+ \cdot 0.85^{\tau_i/60} \cdot \max(0.4, 0.88^k) \cdot \rho_i,$$

with $\rho_{\text{Hal}} = 1.0$ and $\rho_{\text{Baku}} = 0.94$. The engine returns probability zero at $z \geq 300$. The chance branch is therefore

$$\omega \in \{\text{survive}, \text{die}\}, \quad \mathbb{P}(\omega = \text{survive}) = p_{\text{survive}}.$$

2.5 Utility

All rigorous solver modules use terminal-only utility from Hal's perspective:

$$u_{\text{Hal}}(s) = \begin{cases} +1, & \text{Hal is the terminal winner,} \\ -1, & \text{Baku is the terminal winner,} \\ 0, & \text{draw or neutral terminal state.} \end{cases}$$

Nonterminal frontier states are not shaped in the exact solver. They are recorded as unresolved probability mass when the finite horizon is exhausted.

3 Game-Theoretic Foundation

Assumption 1 (Fixed engine model). *The Python engine is treated as the rules oracle. The solver is correct only relative to this engine's transition and legality functions.*

Definition 1 (Finite-horizon value). *Let $V_h(s)$ be Hal's value at state s with h half-rounds remaining in the exact recursion. If s is terminal, then $V_h(s) = u_{\text{Hal}}(s)$. If $h = 0$ and s is nonterminal, then $V_0(s) = 0$ and the unresolved mass is 1.*

For nonterminal s and $h > 0$, form a Hal-payoff matrix $M_h(s)$ over the legal exact seconds:

$$M_h(s)_{d,c} = \mathbb{E}_{\omega \sim P(\cdot|s,d,c)} [V_{h-1}(T(s,d,c,\omega))].$$

If Hal is the dropper, the row player maximizes M_h ; if Hal is the checker, the signs are transposed so the row-player LP is still a maximin solve.

The implemented row-player LP is the standard finite zero-sum matrix-game program:

$$\begin{aligned} \max_{\sigma, v} \quad & v \\ \text{s.t.} \quad & \sum_i \sigma_i M_{ij} \geq v \quad \forall j, \\ & \sum_i \sigma_i = 1, \\ & \sigma_i \geq 0. \end{aligned}$$

This appears as `solve_minimax` in `environment/cfr/exact.py`.

Proposition 1 (Exact finite-horizon equilibrium value). *For a fixed engine state s , fixed horizon h , and full legal action sets $A_D(s), A_C(s)$, `solve_exact_finite_horizon` returns the value of the finite-horizon stochastic zero-sum game induced by the engine and the terminal-only utility.*

Proof. The base case is terminal utility or an explicit unresolved frontier value. For $h > 0$, every joint action is expanded through the engine. If death is possible, the two chance branches are weighted by the referee probability; otherwise the transition has probability one. The resulting $M_h(s)$ is a finite zero-sum matrix game. By von Neumann’s minimax theorem, the LP value equals the equilibrium value of that simultaneous move. Induction on h then gives the finite-horizon stochastic-game value. $\square$

The proposition is intentionally narrow. It proves the exact finite-horizon solver, not the learned solver. The learned solver inherits strength only through approximation quality and validation.

4 Exact Core and Firewall

The rigorous namespace `environment/cfr/` is guarded by tests that prevent bucketed abstractions, reward shaping, observations, and route labels from leaking into exact search. This is important: once shaped reward or a bucket representative enters the exact core, the LP value is no longer a statement about the canonical engine game.

The exact core currently provides:

- **ExactPublicState**: a full-precision public state key for cache and tablebase comparisons.
- **ExactGameSnapshot**: mutation-safe snapshot/restore around the authoritative engine.
- **enumerate_joint_actions**: full exact-second action enumeration.
- **expand_joint_action**: engine-derived chance expansion.
- **solve_exact_finite_horizon**: recursive exact minimax with unresolved frontier accounting.
- A bounded LRU cache keyed by public state, horizon, and perspective.

The latest cache change matters operationally. Phase G’s wide LP grid would otherwise duplicate deep recursion caches across worker processes and exhaust memory. The current default cap is 30,000 entries per worker, controlled by `STL_SOLVE_CACHE_MAXSIZE`.

5 Selective Search and Tablebase

Full-width exact search is expensive: normal turns have up to 60×60 joint actions and leap turns can include a 61st drop action for Baku. At horizon 3 the tree becomes too large for broad corpus generation.

`environment/cfr/selective.py` defines a candidate generator that keeps:

$$\{1, 2, 58, 59, 60, 61\}$$

where legal, plus local diagonals, safe-check boundaries, and overflow threshold boundaries. The central candidate quantities are

$$\text{overflow}(x_C) = 300 - \lfloor x_C \rfloor$$

and

$$\text{safe}(x_C) = 299 - \lfloor x_C \rfloor.$$

Selective search solves the same minimax recursion over this reduced candidate set and can audit against full-width exact search on small states.

The tablebase registry in `environment/cfr/tablebase.py` stores tactical scenarios with pinned expected values or relational properties. Pinned entries are exact ground-truth anchors for training and calibration. Phase F expanded this registry from a tiny set to a broader suite of forced-overflow, clock, leap-window, fatigue, TTD, asymmetric-death, and double-overflow cases.

6 Matrix-Game MCTS

The MCTS layer is not a vanilla single-agent UCT tree. Each node is itself a simultaneous-move matrix game. A node stores candidate drop seconds, candidate check seconds, a value matrix Q , visit counts N , a prior matrix P , and transposition links.

At selection time it forms

$$\tilde{Q}_{ij} = Q_{ij} + c_{\text{puct}} P_{ij} \frac{\sqrt{N_{\text{node}}}}{1 + N_{ij}},$$

then solves the matrix game over $\tilde{Q}$ and samples a joint action from the resulting dropper/checker strategies. The engine is stepped into the child; if a death is possible, MCTS samples the survival branch. At an unexpanded nonterminal leaf, the supplied evaluator returns a Hal-perspective value and optional dropper/checker policy priors. Backup uses a running mean and does not flip signs, because every value is already in Hal’s perspective.

The current MCTS implementation includes:

- evaluator-supplied policy priors over 61 dropper and 61 checker seconds;
- transposition reuse keyed by exact public state;
- principal-line extraction via most-visited cells;
- optional root subgame re-solving at critical states.

7 Value Network

The current neural evaluator in `hal/value_net.py` uses 23 scalar features: normalized cylinders, total time dead, death counts, safe budgets, projected fail-survival probabilities, role bit, normalized clock, round/half features, LSR variation one-hot features, leap-turn bit, rounds until the leap window, referee CPR count, active LSR bit, and leap proximity.

The network is:

$$\phi_{\theta}(x) = \text{ReLU}(W_2 \text{ReLU}(W_1 x + b_1) + b_2),$$

with a scalar value head

$$\hat{v}_{\theta}(x) = \tanh(w_v^{\top} \phi_{\theta}(x) + b_v)$$

and a policy head with 122 logits: 61 for dropper seconds and 61 for checker seconds. The historical default is hidden width 64. Phase I-1 uses hidden width 128, giving about 35.5K parameters, up from about 13.7K.

Training minimizes source-weighted value MSE plus masked policy cross-entropy:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{m=1}^n \alpha_{s_m} (\hat{v}_{\theta}(x_m) - y_m)^2 + \lambda [H(\pi_m^D, \hat{\pi}_{\theta}^D) + H(\pi_m^C, \hat{\pi}_{\theta}^C)],$$

where illegal seconds are masked before the softmax. Source weights are used because terminal and tablebase anchors can be numerically rare but strategically decisive.

8 Target Generation and Bootstrap Loop

The repository no longer treats horizon-1 exact labels as sufficient. Horizon-1 labels are too often unresolved and therefore blind to leap-second routing. The current target hierarchy is:

1. terminal state: terminal utility, source `terminal`;
2. pinned tablebase state: exact pin, source `tablebase`;
3. LSR-significant state: exact horizon 3, source `exact_horizon_3`;
4. active-LSR or near-overflow state: exact horizon 2, source `exact_horizon_2`;
5. otherwise: excluded from the exact corpus and left to MCTS bootstrap.

The AlphaZero-style loop is:

1. train a value/policy network on exact and tablebase anchors;
2. run MCTS using that network at leaves;
3. record root MCTS values and root policies as bootstrap labels;
4. merge exact anchors back in to prevent self-referential drift;
5. train the next network;
6. reject the generation if calibration gates fail.

This follows the expert-iteration pattern used by AlphaZero and related search-and-learning systems: search improves the policy/value estimate, then the network distills search output into a cheaper evaluator.

9 Current Empirical Results

The latest documented chain is Phase G + I-1 through Phase G iteration 3.

Milestone	Held-out overall MSE	Comment
gen-2 v2 baseline	0.10841	Pre-Phase-G best
gen-3 v2	0.132	Plateau/regression
Phase G + I-1	0.070	Strict per-source gate passed
Phase G iter 2	0.06000	Policy-improvement signal remains
Phase G iter 3	0.05934	Converged under current data/architecture

The Phase G LP corpus had 7,168 candidate states and emitted 4,812 records: 2,938 exact-horizon-3, 1,871 exact-horizon-2, and 3 direct tablebase records. After anchors and bootstrap merge, iteration 2 and iteration 3 trained on 6,357-record corpora with 864 MCTS-bootstrap labels plus exact/tablebase and terminal anchors.

The strict held-out gate checks per-source MSE, tablebase MSE, required source presence, and unresolved probability. Phase G iteration 3 passed with:

Source	n	MSE
terminal	24	0.498
tablebase	19	0.007
exact_horizon_2	128	0.012
exact_horizon_3	64	0.004

This is strong evidence that the supervised loop is now data-limited under the current target distribution. It is not yet proof that the root game is solved.

10 Relationship to AlphaZero, CFR, and Poker Solvers

The architecture resembles AlphaZero in its search-distillation loop: MCTS guided by a neural value/policy network produces improved targets, and the next generation network learns those targets. The difference is that each STL node is a simultaneous zero-sum matrix game rather than a single-player choice by the side to move.

The architecture resembles CFR systems in its use of equilibrium values for local matrix games and its concern with exploitability, but the current core does not perform regret minimization over information sets. The namespace name `environment/cfr/` is therefore historical and partially aspirational. The rigorous exact solver is better described as finite-horizon stochastic minimax with LP solves at simultaneous-move nodes.

DeepStack, Libratus, Pluribus, Deep CFR, and ReBeL solve harder imperfect-information poker settings by combining abstraction, regret minimization, depth-limited solving, and learned value functions. Those tools are relevant literature, but they are not required if the canonical STL model remains fully public after the legality reduction. If future canon work reintroduces hidden memory or belief states, the correct next framework would be public-belief-state solving or CFR over information sets. Under the current repo contract, that would add complexity without improving the formal model being solved.

11 What Is Rigorous and What Is Approximate

The mathematically rigorous pieces are:

- the engine-derived transition model, assuming the engine is the accepted rule oracle;
- the exact finite-horizon full-width solver;
- LP minimax at each simultaneous move;
- terminal-only utility in the rigorous core;
- pinned tablebase scenarios whose expected values are exact by construction.

The approximate pieces are:

- finite-horizon cutoff and unresolved mass;
- selective candidate generation;
- MCTS sampling variance and finite iteration budget;
- learned value-network approximation error;
- policy-head targets distilled from exact or MCTS root strategies;
- empirical calibration gates, which measure but do not prove global exploitability.

The most important distinction for the white paper is this: a low held-out MSE says the evaluator predicts the current target corpus well. It does not by itself bound exploitability across the full game tree. To make the solver paper-grade, the next version should add explicit exploitability and candidate-set regret audits on increasingly broad state samples.

12 Best Next Steps

12.1 1. Expand the Held-Out and Training Anchors

Phase G iteration 3 appears to have reached a data ceiling. The highest-value next step is Phase F-2: expand the tablebase and held-out registry beyond the current forced-overflow-heavy family. Add pins around:

- non-299 near-overflow thresholds;
- non-leap midgame states where the correct play routes into LSR variation 2 several rounds later;
- asymmetric TTD/death-count states that are not already collapsed by the 23-feature representation;
- role-swapped leap-window states;
- states where Baku’s second-61 drop is legal but not trivially winning.

Do this before increasing model size again. The hidden-128 run already showed that larger capacity without enough diverse data overfits.

12.2 2. Add High-Variance MCTS Reanalysis

Phase H should select states where MCTS value has high seed variance, high policy entropy, or disagreement with exact/selective labels, then re-run those states at higher iteration counts. This is a better use of compute than uniformly enlarging the bootstrap grid.

12.3 3. Quantify Exploitability

The repo has diagnostics for exact matrix games, but the white paper needs a headline exploitability story for the learned agent. Add:

- exact best-response gaps on small full-width subgames;
- selective-vs-full-width candidate regret on audited states;
- MCTS seed confidence intervals at fixed roots;
- value error stratified by state feature axes, not only source class.

12.4 4. Promote Policy Validation

The value head is currently better justified than the policy head. Add policy metrics:

- root-action KL against exact/selective strategies on held-out states;
- illegal-mass checks after masking;
- top-action agreement where the exact equilibrium has low entropy;
- entropy calibration for mixed strategies.

12.5 5. Run Canonical Principal-Line Validation

`scripts/validate_against_manga.py` currently supports the intended validation path, but its default evaluator is still the terminal-only placeholder. The next useful milestone is to load `checkpoints/gen_phaseG_iter3/best.pt`, run the canonical R1T1-to-R9T2 scenario against `BakuLSREngineeringTeacher`, and report the principal line, root values, and deviations from the documented canonical strategy.

12.6 6. Decide Whether This Is a Solver or a Strong Agent

For a paper, terminology matters. The current system is defensibly a calibrated search-and-learning agent with exact local solvers. It is not yet a full-game solved equilibrium strategy unless the paper can bound exploitability over the relevant starting distribution. The next writing standard should be:

- call exact finite-horizon outputs “solves”;
- call Phase G iter 3 a “calibrated evaluator”;
- call MCTS plus evaluator a “search policy”;
- reserve “solver” for the packaged system only after exploitability and candidate-regret bounds are included.

13 Conclusion

The repository now has the right skeleton for a rigorous solver paper: canonical rules, a formal public-state model, engine-derived stochastic transitions, exact finite-horizon minimax, tablebase pins, matrix-game MCTS, value/policy distillation, calibration gates, and reproducible scripts. The main scientific work remaining is to turn empirical calibration into a stronger approximation argument: broader anchors, high-variance reanalysis, explicit exploitability estimates, and principal-line validation on the canonical scenario.

References

- [1] John von Neumann. Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen*, 100:295–320, 1928. <https://doi.org/10.1007/BF01448847>
- [2] Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953. <https://doi.org/10.1073/pnas.39.10.1095>
- [3] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. *Advances in Neural Information Processing Systems*, 2007. https://papers.nips.cc/paper_files/paper/2007/hash/08d98638c6fcd194a4b1e6992063e944-Abstract.html
- [4] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012. <https://doi.org/10.1109/TCIAIG.2012.2186810>
- [5] Matej Moravcik, Martin Schmid, Neil Burch, Viliam Lisy, Dustin Morrill, Nolan Bard, Trevor Davis, Kevin Waugh, Michael Johanson, and Michael Bowling. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, 2017. <https://doi.org/10.1126/science.aam6960>
- [6] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. <https://doi.org/10.1126/science.aar6404>
- [7] Noam Brown and Tuomas Sandholm. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018. <https://doi.org/10.1126/science.aao1733>
- [8] Noam Brown and Tuomas Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456):885–890, 2019. <https://doi.org/10.1126/science.aay2400>
- [9] Noam Brown, Adam Lerer, Sam Gross, and Tuomas Sandholm. Deep counterfactual regret minimization. *Proceedings of the 36th International Conference on Machine Learning*, 2019. <https://proceedings.mlr.press/v97/brown19b.html>

- [10] Noam Brown, Anton Bakhtin, Adam Lerer, and Qucheng Gong. Combining deep reinforcement learning and search for imperfect-information games. *Advances in Neural Information Processing Systems*, 2020. <https://proceedings.neurips.cc/paper/2020/hash/c61f571dbd2fb949d3fe5ae1608dd48b-Abstract.html>